universität wien

MULTIOMICS DATA SCIENCE - CLUSTERING

CLUSTERING

- Method for partitioning of data objects (points, etc …) in independent groups

- Two different strategies:
  - Hard clustering: data objects are assigned to exactly one group

  - Soft clustering: for each data object, an assignment probability is computed to every group

DIFFERENCE BETWEEN CLUSTERING AND CLASSIFICATION

- For clustering, features of the observations are used, to group the observations
- Clustering is also called **unsupervised learning**

DIFFERENCE BETWEEN CLUSTERING AND CLASSIFICATION

- For clustering, features of the observations are used, to group the observations
- Clustering is also called **unsupervised learning**
- For a classification, both the features and the nominal target variable are used
- A model is computed, which translates a set of features into the target variables
- Classification is also called **supervised learning**

DIFFERENT ANALYSIS METHODS FOR CLUSTERING

- We distinguish between hierarchical and partitioning clustering

- Hierarchical approaches set all data objects in relation to each other, either distance based or feature based:
  - Agglomerative approach: At the beginning, each data object is in its own cluster (singleton). The clusters are merged together step by step (**bottom-up**, commonly used)

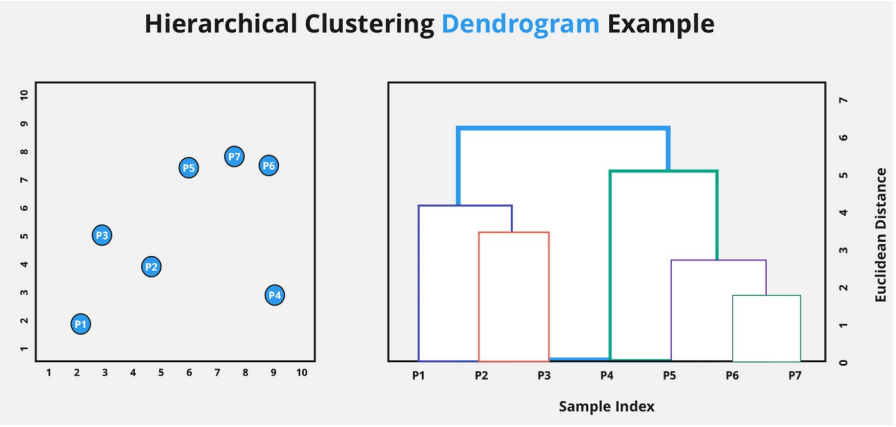
  - Divisive approach: An initial global cluster is being split in each step in several sub clusters (**top-down**)

HIERARCHICAL CLUSTERING

- For a hierarchical clustering, we need numerical data and a distance measure

- In this example, our numerical values are the x/y coordinates and the distance measure is the Euklidean distance

- Different distance measures:
  - L_n-Norm
  - Pearson
  - Mahalanobis



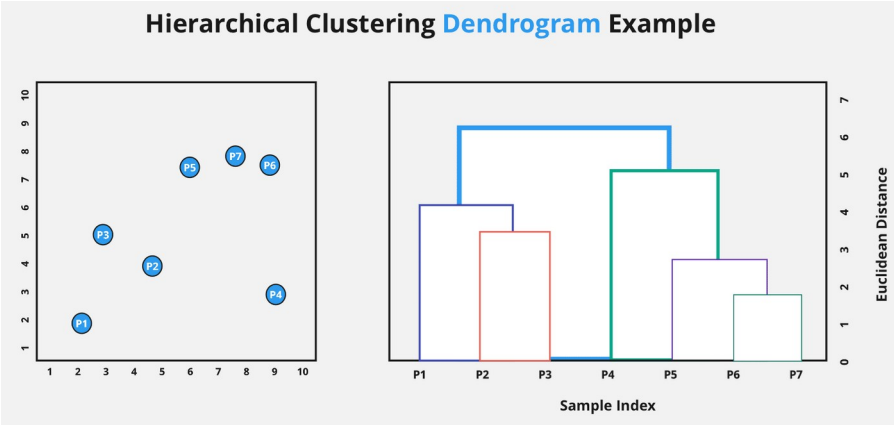
Quelle: *https://dataaspirant.com*

AGGLUMERATIVE CLUSTERING

- First all data points are in separate clusters

- As long as more than one cluster exist:
  - Find the cluster pair with the lowest distance

  - Merge the cluster pair to one cluster



Quelle: *https://dataaspirant.com*

universität wien

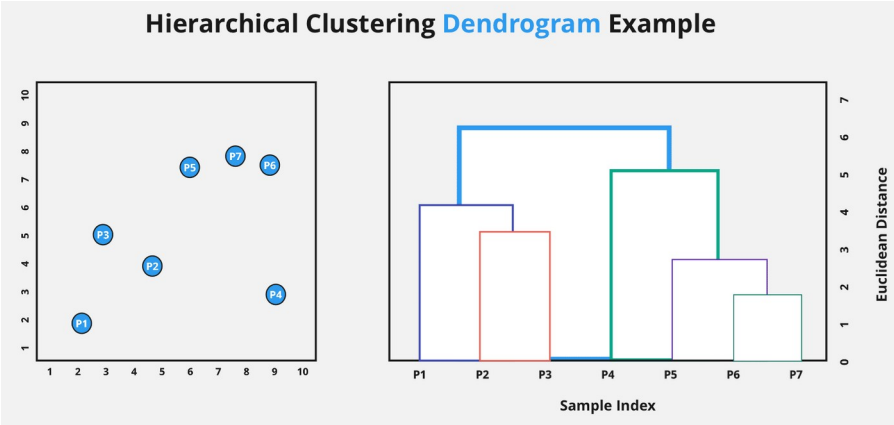
AGGLUMERATIVE CLUSTERING

- First all data points are in separate clusters

- As long as more than one cluster exist:
  - Find the cluster pair with the lowest distance

  - Merge the cluster pair to one cluster

- What is the distance between the cluster (P1, P2, P3) and (P4, P5, P6, P7)?

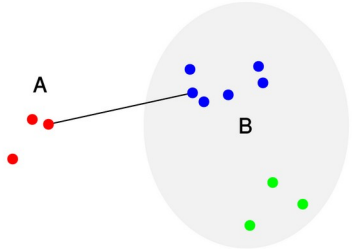


Quelle: *https://dataaspirant.com*

universität wien

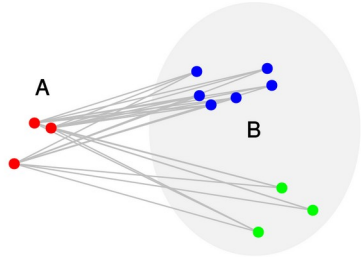
LINKAGE STRATEGIES

Single linkage (min. distance)



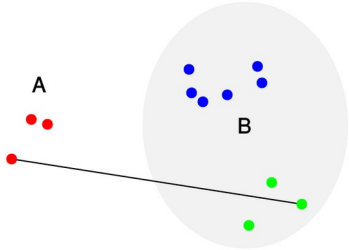
Quelle: *https://de.wikipedia.org*

Average-Linkage (Avg. of all distances):



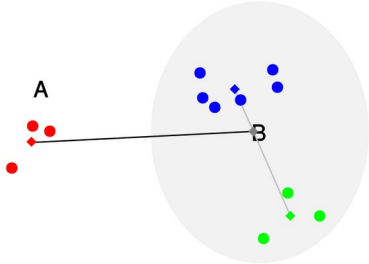
Quelle: *https://de.wikipedia.org*

Complete linkage (max. distance)



Quelle: *https://de.wikipedia.org*

Centroid-Linkage (Distance to cluster center)



Quelle: *https://de.wikipedia.org*

universität wien

LINKAGE STRATEGIES

- Which linkage strategy is the best?
- Single linkage creates more flat dendrograms
- Complete linkage creates rather deep dendrograms

LINKAGE STRATEGIES

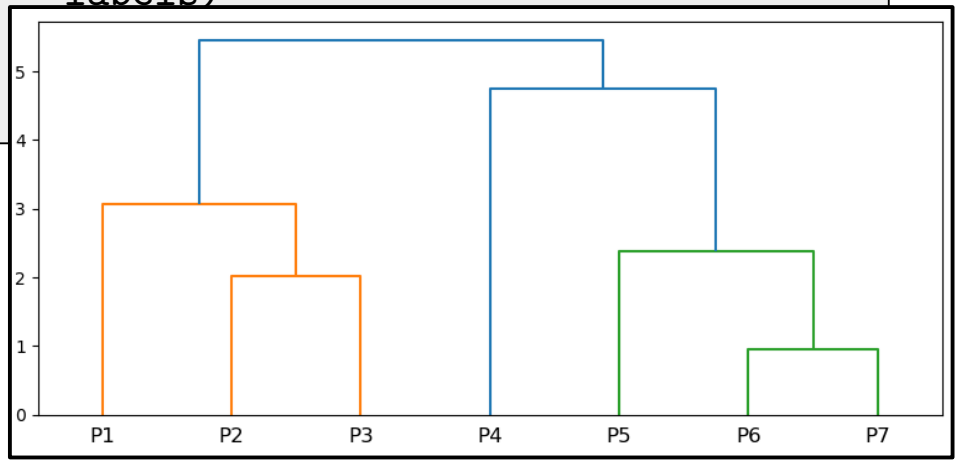
- Which linkage strategy is the best?
- Single linkage creates more flat dendrograms
- Complete linkage creates rather deep dendrograms
- There is no „right“ or „wrong“
- Maybe it is possible to infer the linkage strategy from the nature of the data

HIERARCHICAL CLUSTERING IN PYTHON

```
import numpy as np
from scipy.cluster import hierarchy
import matplotlib.pyplot as plt

x = np.array([[2., 2.], [4.7, 3.9], [3., 5.], [9., 3.], [6., 7.3], [8.8,
7.5], [7.9, 7.8]])
labels = ["P1", "P2", "P3", "P4", "P5", "P6", "P7"]
temp = hierarchy.linkage(x, method = 'centroid', metric = "euclidean")
dn = hierarchy.dendrogram(temp, labels = labels)

plt.show()
```

HIERARCHICAL CLUSTERING IN PYTHON

```
import numpy as np
from scipy.cluster import hierarchy
import matplotlib.pyplot as plt

x = np.array([[2., 2.], [4.7, 3.9], [3., 5.], [9., 3.], [6., 7.3], [8.8,
7.5], [7.9, 7.8]])
labels = ["P1", "P2", "P3", "P4", "P5", "P6", "P7"]
temp = hierarchy.linkage(x, method = 'centroid', metric = "euclidean")
dn = hierarchy.dendrogram(temp, labels = labels)

plt.show()
```



universität wien

METHODS AND METRICS

- Methods:
  - Single linkage (`single`), def.
  - Complete linkage (`complete`)
  - Average linkage (`average`)
  - Centroid linkage (`controid`)

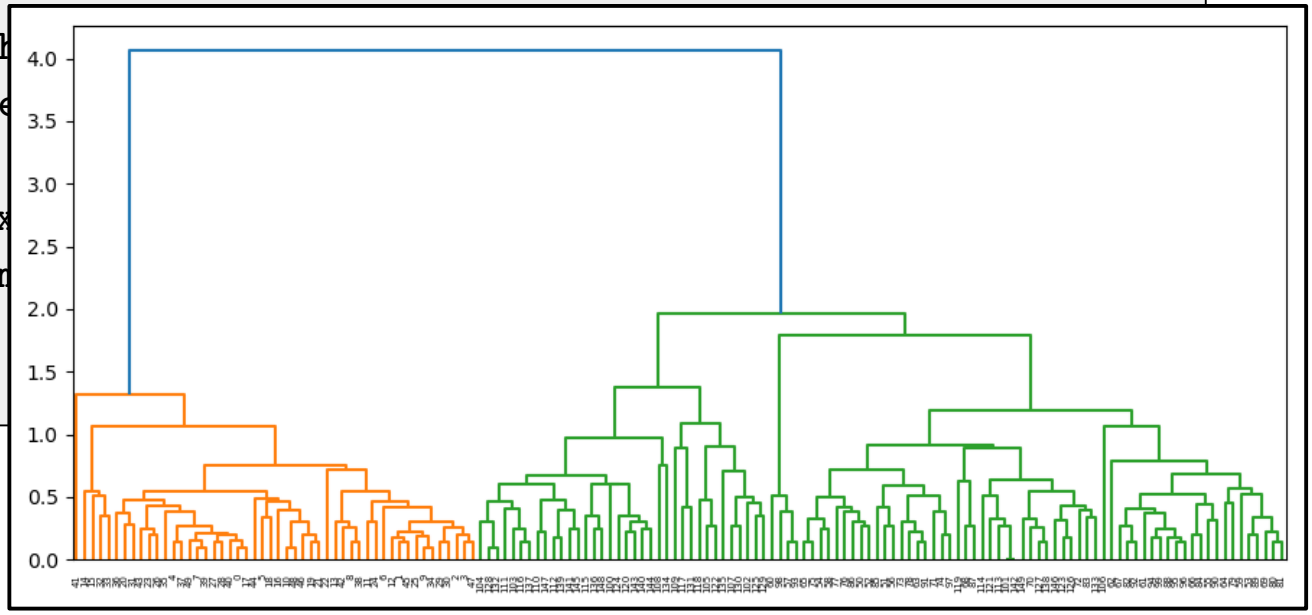
- Metrics:
  - L_1 Norm (`cityblock`)
  - L_2 Norm (`euclidean`), default
  - Mahalanobis (`mahalanobis`)
  - Cosine Similarity (`cosine`)
  - Pearson correlation (`correlation`)

EXAMPLE WITH THE IRIS-DATASET

```
from scipy.cluster import hierarchy
import matplotlib.pyplot as plt
import pandas as pd

iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
x, y = iris.loc[:,features], iris["species"]

lkg = hierarchy.linkage(x, method = 'average', metric = "euclidean")
dn = hierarchy.dendrogram(lkg)

plt.show()
```

EXAMPLE WITH THE IRIS-DATASET

```
from scipy.cluster import hierarchy
import matplotlib.pyplot as plt
import pandas as pd

iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length
x, y = iris.loc[:,feature

lkg = hierarchy.linkage(x
dn = hierarchy.dendrogram

plt.show()
```



universität wien

EXAMPLE WITH THE IRIS-DATASET

```
lkg = hierarchy.linkage(x, method = 'average', metric = "euclidean")
dn = hierarchy.dendrogram(lkg)

print(dn["leaves"])      # Order of the data points in the dendrogram
[41, 14, 15, 32, 33, 36, 20, 31, 43, 23, 26, 35, 4, 37, 49, 7, 39, 27, 28,
40, 0, 17, 44, 5, 18, 16, 10, 48, 46, 19, 21, 22, 13, 42, 8, 38, 11, 24, 6,
12, 1, 45, 25, 9, 34, 29, 30, 2, 3, 47, 104, 128, 132, 111, 103, 116, 137,
110, 147, 112, 139, 141, 145, 115, 136, 148, 100, 124, 120, 143, 140, 144,
108, 134, 109, 117, 131, 118, 105, 122, 135, 107, 130, 102, 125, 129, 60,
98, 57, 93, 65, 75, 54, 58, 77, 76, 86, 50, 52, 85, 51, 56, 73, 78, 63, 91,
71, 74, 97, 119, 68, 87, 114, 121, 113, 101, 142, 149, 70, 127, 138, 146,
123, 126, 72, 83, 133, 106, 62, 67, 82, 92, 61, 94, 99, 88, 95, 96, 66, 84,
55, 90, 64, 79, 59, 53, 89, 69, 80, 81]
```

EXAMPLE WITH THE IRIS-DATASET / CLUSTERING

```
lkg = hierarchy.linkage(x, method = 'average', metric = "euclidean")
cluster_index = hierarchy.fcluster(lkg, t = 1.8, criterion = 'distance')

print(cluster_index)      assignment of data points to cluster index
[1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3
 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 2 3 2 2 2 2 3 2 2 2 2
 2 2 3 3 2 2 2 2 3 2 3 2 3 2 2 3 3 2 2 2 2 2 3 2 2 2 2 3 2 2 2 3 2 2 2 3 2
 2 3]
```

universität wien

EXAMPLE WITH THE IRIS-DATASET / CLUSTERING

```
lkg = hierarchy.linkage(x, method = 'average', metric = "euclidean")
cluster_index = hierarchy.fcluster(lkg, t = 1.8, criterion = 'distance')

# create a dictionary: cluster index  → list of data point indexes
clusters = {c: [] for c in set(cluster_index)}
for i, c in enumerate(cluster_index): clusters[c].append(i)

print(clusters)
{1: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19,
20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38,
39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49], 2: [100, 102, 103, 104, 105,
107, 108, 109, 110, 111, 112, 115, 116, 117, ...]}
```

EXAMPLE WITH THE IRIS-DATASET USING A PCA

```
from sklearn.decomposition import PCA

iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
x, y = iris.loc[:,features], iris["species"]

for feat in features:                    # standardization
    x.loc[:, feat] = (x[feat] - x[feat].mean()) / x[feat].std()
xt = PCA().fit_transform(x)[:, :2]    # PCA and reduction to 2 components

# clustering
lkg = hierarchy.linkage(xt, method = 'average', metric = "euclidean")
dn = hierarchy.dendrogram(lkg)
```

EXAMPLE WITH THE IRIS-DATASET USING A PCA

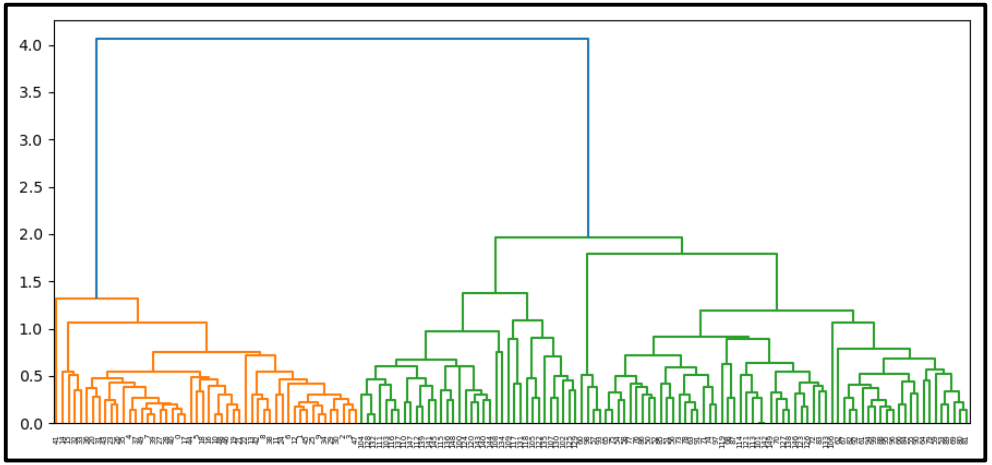
```
from sklearn.decomposition import PCA

iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
x, y = iris.loc[:,features], iris["species"]

for feat in features:                    # standardization
    x.loc[:, feat] = (x[feat] - x[feat].mean()) / x[feat].std()
xt = PCA().fit_transform(x)[:, :2]    # PCA and reduction to 2 components

# clustering
lkg = hierarchy.linkage(xt, method = 'average', metric = "euclidean")
dn = hierarchy.dendrogram(lkg)
```

universität wien

Who decided at which dendrogram distance the data points are clustered together?

EXAMPLE WITH THE IRIS-DATASET

- Hierarchical clustering does NO partitioning!!!
- Algorithm (somehow) decided to cut above a distance of 2.0

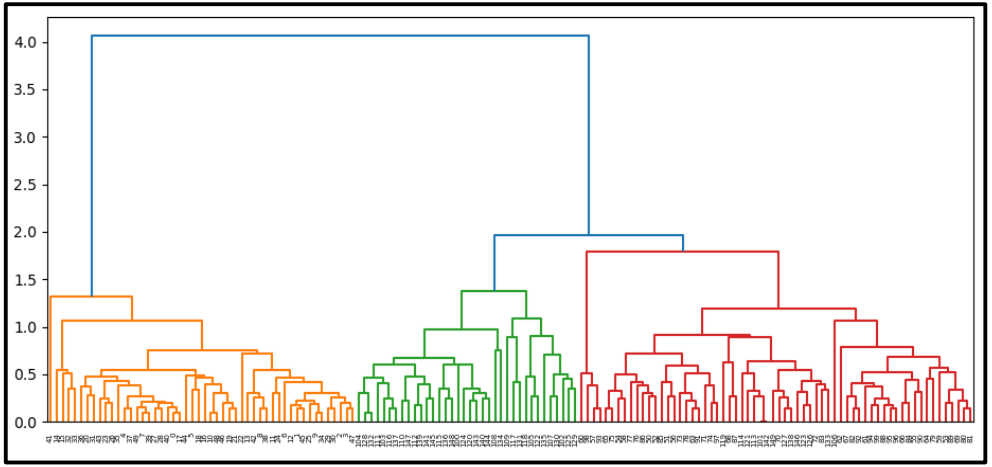


universität wien

EXAMPLE WITH THE IRIS-DATASET

- Hierarchical clustering does NO partitioning!!!
- Algorithm (somehow) decided to cut above a distance of 2.0
- We can set a threshold

```
dn = hierarchy.dendrogram(lkg, color_threshold = 1.8)
```



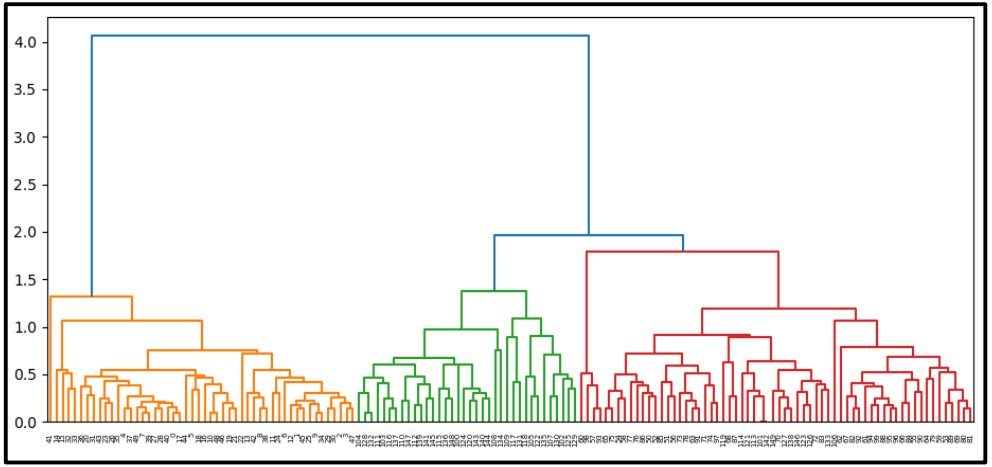
universität wien

EXAMPLE WITH THE IRIS-DATASET

- Hierarchical clustering does NO partitioning!!!
- Algorithm (somehow) decided to cut above a distance of 2.0
- We can set a threshold

```
dn = hierarchy.dendrogram(lkg, color_threshold = 1.8)
```

- Threshold has to be set manually



universität wien

K-MEANS FOR PARTITIONING CLUSTERING

- Another scenario: we know, that the data points are equally distributed in all dimensions around their cluster center

- For a given cluster number, the algorithmus should now compute an actual partitioning

K-MEANS FOR PARTITIONING CLUSTERING

- Another scenario: we know, that the data points are equally distributed in all dimensions around their cluster center

- For a given cluster number, the algorithmus should now compute an actual partitioning

- K-Means is often used, when the cluster number is known, the variance within the clusters is low and clusters of of equal size

- Technically, K-Means tries to minimize the distances of all data points to their centers, let *k* be the number of clusters, S_i a cluster and x_j a data point within S_i:

$$\min \sum_{i}^{k} \sum_{x_j \in S_i} \|x_j - \mu_i\|^2$$

K-MEANS EXAMPLE WITH THE IRIS-DATASET

```
from sklearn.cluster import KMeans
import pandas as pd

iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
x = iris.loc[:,features]

kmeans = KMeans(n_clusters = 3)
kmeans.fit(x)
print(kmeans.labels_)    # Assignment of data point to cluster
[1 1 1 1 1 1 1 1 1 1 1 1 1 ... 0 0 0 2 0 0 0 2 0 0 0 2 0 0 2]
```

K-MEANS EXAMPLE WITH THE IRIS-DATASET

```
from sklearn.cluster import KMeans
import pandas as pd

iris = pd.read_csv('https://raw.githubusercontent.com/mwaskom/seaborn-data/
master/iris.csv')
features = ["sepal_length", "sepal_width", "petal_length", "petal_width"]
x = iris.loc[:,features]

kmeans = KMeans(n_clusters = 3)
kmeans.fit(x)
print(kmeans.labels_)    # Assignment of data point to cluster
[1 1 1 1 1 1 1 1 1 1 1 1 1 ... 0 0 0 2 0 0 0 2 0 0 0 2 0 0 2]
```

Can we see the clustering?

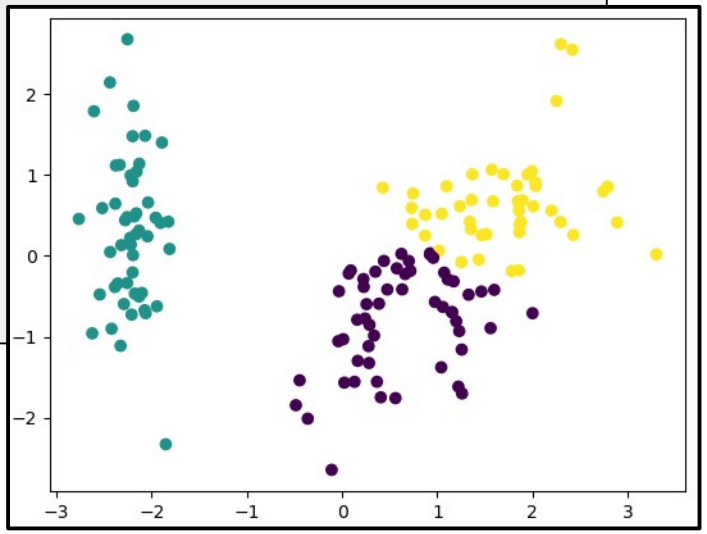
Yes, but no, but yes, but not really with 4 dimensions!

K-MEANS EXAMPLE WITH THE PCA OF THE IRIS-DATASETS

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

x = iris.loc[:,features]
for feat in features:                    # standardization
    x.loc[:, feat] = (x[feat] - x[feat].mean()) / x[feat].std()
xt = PCA().fit_transform(x)[:,:2]        # PCA and reduction to 2 components

kmeans = KMeans(n_clusters = 3)
kmeans.fit(xt)                           # clustering

plt.scatter(xt[:,0], xt[:,1], c = kmeans.labels_)
plt.show()
```

K-MEANS EXAMPLE WITH THE PCA OF THE IRIS-DATASETS

```
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

x = iris.loc[:,features]
for feat in features:                    # standardization
    x.loc[:, feat] = (x[feat] - x[feat].mean()) / x[feat].std()
xt = PCA().fit_transform(x)[:,:2]        # PCA and reduction to 2 components

kmeans = KMeans(n_clusters = 3)
kmeans.fit(xt)                           # clustering

plt.scatter(xt[:,0], xt[:,1], c = kmeans.labels_)
plt.show()
```



universität wien

UNKNOWN NUMBER OF CLUSTERS

- A common problem is, that the cluster number is not known upfront
- In this case we can cheat a little by running K-Means several times with incrementing cluster number and monitor the global error

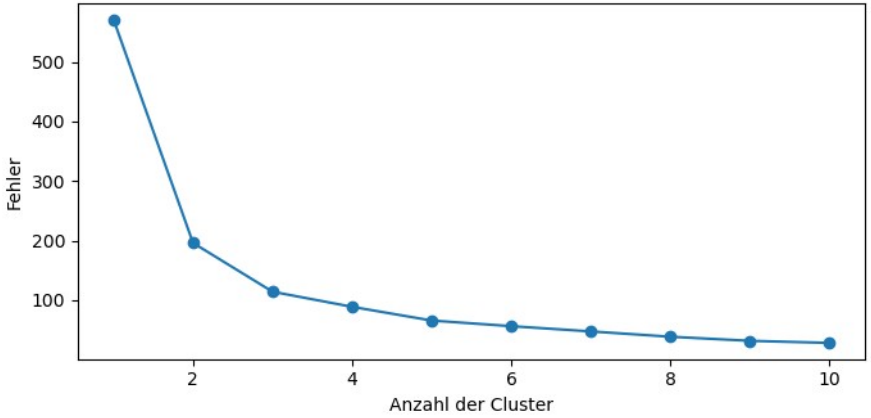
UNKNOWN NUMBER OF CLUSTERS

- A common problem is, that the cluster number is not known upfront
- In this case we can cheat a little by running K-Means several times with incrementing cluster number and monitor the global error

```
inertias = []
for i in range(1,11):    # cluster size from 1 to 10 (incl.)
    inertias.append(KMeans(n_clusters=i).fit(xt).inertia_)

plt.plot(range(1,11), inertias, marker='o')
plt.xlabel('Anzahl der Cluster')
plt.ylabel('Fehler')
plt.show()
```

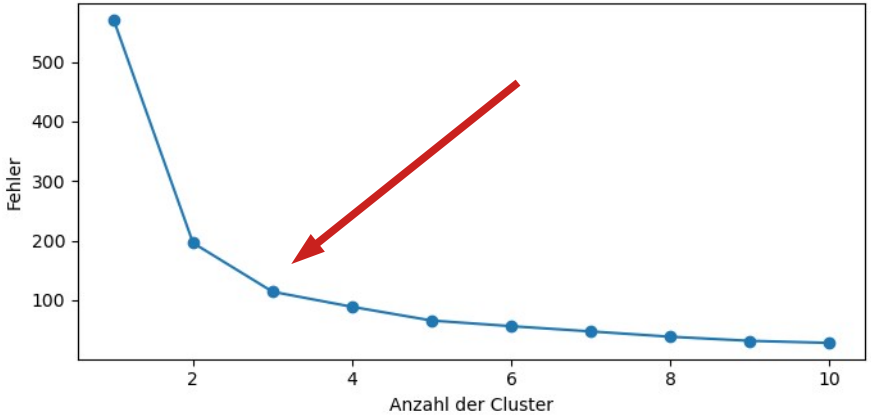
UNKNOWN NUMBER OF CLUSTERS

- A common problem is, that the cluster number is not known upfront
- In this case we can cheat a little by running K-Means several times with incrementing cluster number and monitor the global error



UNKNOWN NUMBER OF CLUSTERS

- A common problem is, that the cluster number is not known upfront
- In this case we can cheat a little by running K-Means several times with increm-enting cluster number and monitor the global error
- A „good“ approach for estimating the cluster size is to look at which size the curve starts to go linearly
- When the cluster size equals the number of data points, then the error = 0



CLUSTERING WITH UNKNOWN CLUSTER SIZE

- It‘s a general problem for many clustering algorithms that they need a static cluster number as input parameter

- If the exact cluster number is not known upfront, then we need to trial & error with a hierarchical clustering or K-Means to estimate the cluster number

CLUSTERING WITH UNKNOWN CLUSTER SIZE

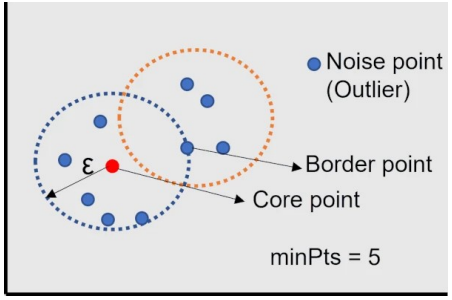
- It‘s a general problem for many clustering algorithms that they need a static cluster number as input parameter

- If the exact cluster number is not known upfront, then we need to trial & error with a hierarchical clustering or K-Means to estimate the cluster number

- DBSCAN is an extended algorithm, which is clustering data points according to the density of their neighbors without a prior determined cluster number

DBSCAN

- **D**ensity **B**ased **S**patial **C**lustering of **A**pplications with **N**oise
- Examination of neighbors of a point:
- Beginning from a starting point, all neighbors of the point are put into the same cluster which are within a defined range



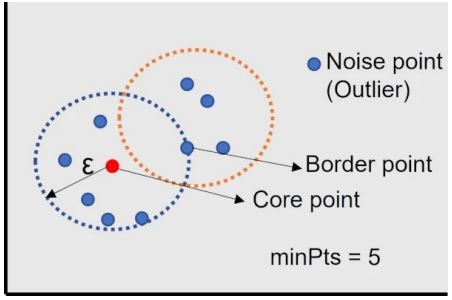
Quelle: *https://www.reneshbedre.com*

DBSCAN

- **D**ensity **B**ased **S**patial **C**lustering of **A**pplications with **N**oise

- Examination of neighbors of a point:

- Beginning from a starting point, all neighbors of the point are put into the same cluster which are within a defined range



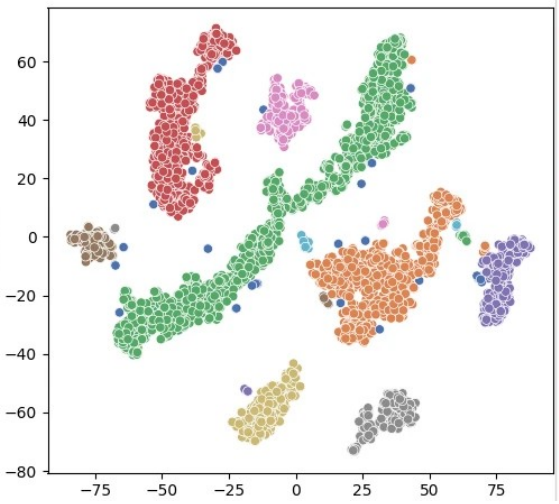
Quelle: *https://www.reneshbedre.com*

- Afterwards, the neighbors are visited and the same steps are executed

- When all neighbors are visited and no unvisited point can be reached, a new cluster will be opened for the first unvisited point

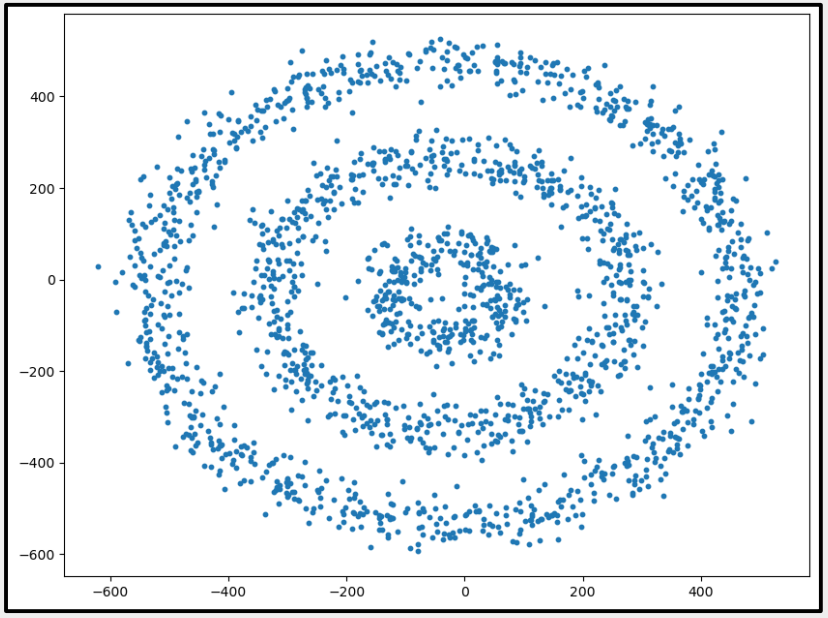
DBSCAN

- The properties of DBSCAN are:
  - Can cope with clusters, that follow no strict pattern (random cluster form)
  - It needs information for the maximum distance
  - It needs information for the minimum number of points within a cluster



EXAMPLE FOR DBSCAN

```
# df is a dataframe with x, y coordinates
plt.scatter(df[‘x‘], df[‘y‘])
plt.show()
```

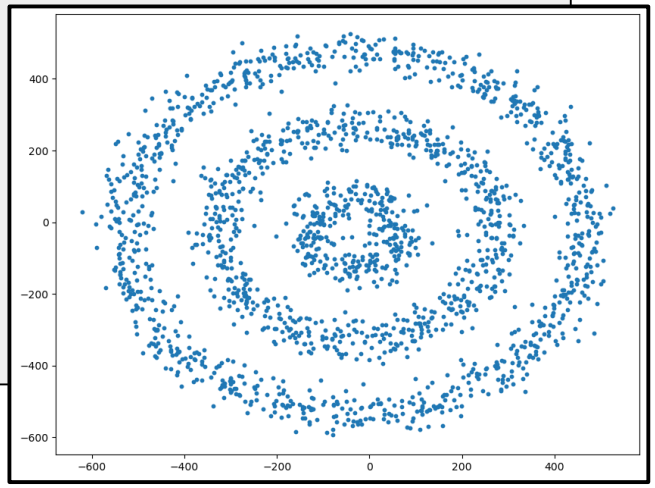


EXAMPLE FOR DBSCAN

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN()
dbscan.fit(df)
print(dbscan.labels_)
[-1 -1 -1 ... -1 -1 -1]
```

EXAMPLE FOR DBSCAN

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN()
dbscan.fit(df)
print(dbscan.labels_)
[-1 -1 -1 ... -1 -1 -1]

plt.scatter(df[‘x‘], df[‘y‘], c = dbscan.labels_)
plt.show()
```

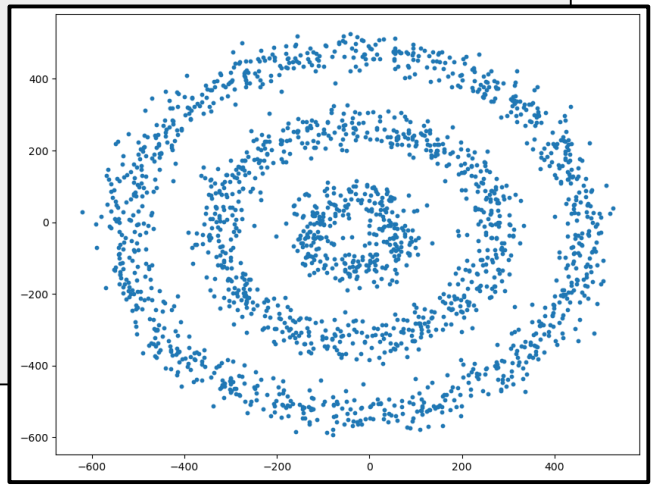


universität wien

EXAMPLE FOR DBSCAN

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN()
dbscan.fit(df)
print(dbscan.labels_)
[-1 -1 -1 ... -1 -1 -1]

plt.scatter(df[‘x‘], df[‘y‘], c = dbscan.labels_)
plt.show()
```

The maximum neighbor distance was nowhere provided!



universität wien

DBSCAN – DETERMINE THE DISTANCE

- For DBSCAN, we also can determine with trial & error a „good“ distance, that fits the input data

- But there is a clever way to find a proper value by considering the distance distributions of all data points to their neighbors

DBSCAN – DETERMINE THE DISTANCE

- Euklidean distance of all points to all points

```
from scipy.spatial.distance import pdist
dist = pdist(df, "euclidean")
print(dist, dist.shape)
[32.54042444 108.596836  9.42709764 ... 56.48938125 31.05904925
 75.94295296] (1999000,)
```

DBSCAN – DETERMINE THE DISTANCE

- Euklidean distance of all points to all points

```
from scipy.spatial.distance import pdist, squareform
dist = pdist(df, "euclidean")
dist = squareform(dist)
print(dist, dist.shape)
[[  0.          32.54042444 108.596836   ... 441.37131923 469.12535743]
 [ 32.54042444   0.          84.3851577  ... 414.73584616 439.82817096]
 ...
 [441.37131923 414.73584616 416.23732466 ...   0.          75.94295296]
 [469.12535743 439.82817096 426.68365105 ...  75.94295296   0.        ]]
(2000, 2000)
```

DBSCAN – DETERMINE THE DISTANCE

- Euklidean distance of all points to all points

```
from scipy.spatial.distance import pdist, squareform
dist = pdist(df, "euclidean")
dist = squareform(dist) + 1e100 * np.identity(df.shape[0])
print(dist)
[[  1.000e+100  32.54042444 108.596836   ... 441.37131923 469.12535743]
 [ 32.54042444   1.000e+100  84.3851577  ... 414.73584616 439.82817096]
 ...
 [441.37131923 414.73584616 416.23732466 ...   1.000e+100  75.94295296]
 [469.12535743 439.82817096 426.68365105 ...  75.94295296   1.000e+100]]
```

DBSCAN – DETERMINE THE DISTANCE

- Minimal Euklidean distance of all points to their direct neighbor:

```
from scipy.spatial.distance import pdist, squareform
dist = pdist(df, "euclidean")
dist = squareform(dist) + 1e100 * np.identity(df.shape[0])
min_dist = dist.min(axis = 0)
print(min_dist)
[ 3.0982324  13.25139874  9.47378351 ...  1.43763974  6.71381454
  5.87071258]
```

DBSCAN – DETERMINE THE DISTANCE

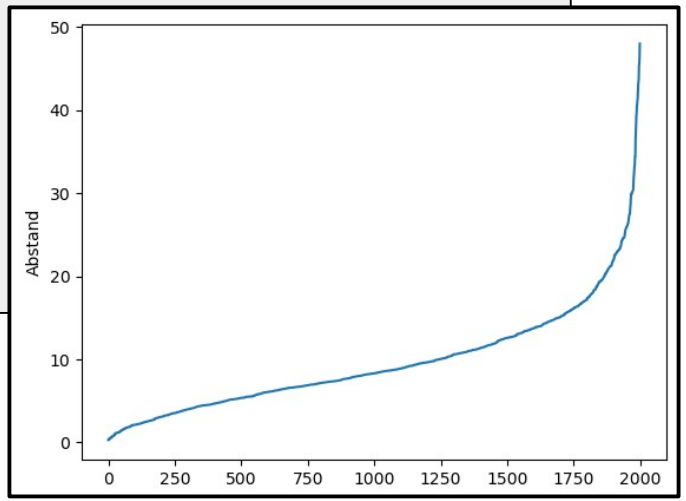
- Minimal Euklidean distance of all points to their direct neighbor:

```
from scipy.spatial.distance import pdist, squareform
dist = pdist(df, "euclidean")
dist = squareform(dist) + 1e100 * np.identity(df.shape[0])
min_dist = dist.min(axis = 0)
print(min_dist)
[ 3.0982324  13.25139874  9.47378351 ...  1.43763974  6.71381454
  5.87071258]

plt.scatter(range(df.shape[0]), sorted(min_dist))
plt.show()
```

DBSCAN – DETERMINE THE DISTANCE

- Minimal Euklidean distance of all points to their direct neighbor:

```
from scipy.spatial.distance import pdist, squareform
dist = pdist(df, "euclidean")
dist = squareform(dist) + 1e100 * np.identity(df.shape[0])
min_dist = dist.min(axis = 0)
print(min_dist)
[ 3.0982324  13.25139874  9.47378351 ...  1.43763974  6.71381454
  5.87071258]

plt.scatter(range(df.shape[0]), sorted(min_dist))
plt.show()
```

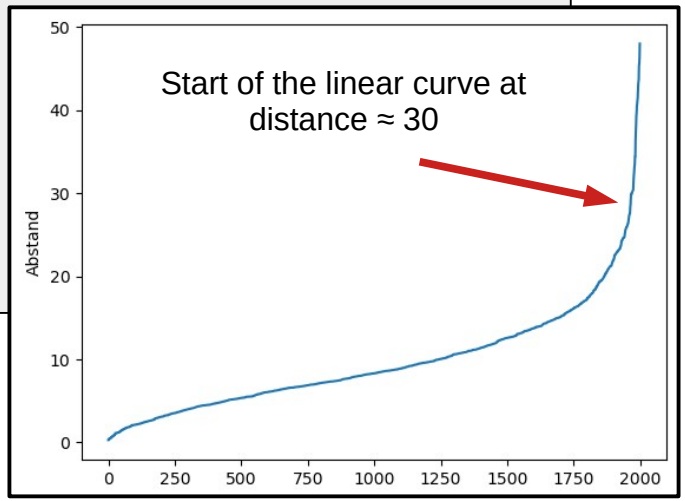


universität wien

DBSCAN – DETERMINE THE DISTANCE

- Minimal Euklidean distance of all points to their direct neighbor:

```
from scipy.spatial.distance import pdist, squareform
dist = pdist(df, "euclidean")
dist = squareform(dist) + 1e100 * np.identity(df.shape[0])
min_dist = dist.min(axis = 0)
print(min_dist)
[ 3.0982324  13.25139874  9.47378351 ...  1.43763974  6.71381454
  5.87071258]

plt.scatter(range(df.shape[0]), sorted(min_dist))
plt.show()
```



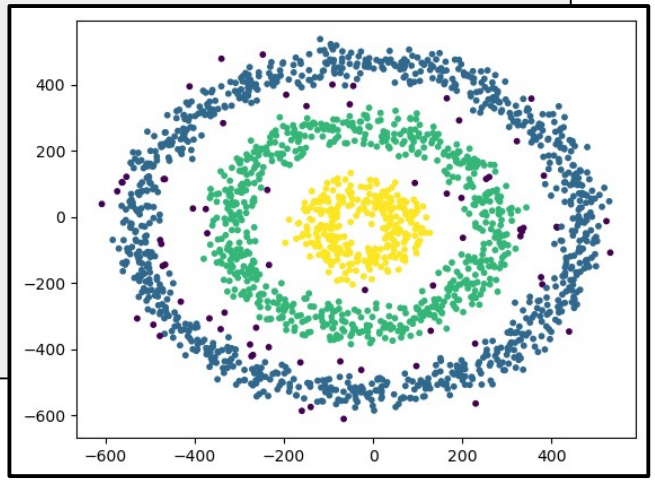
universität wien

EXAMPLE FOR DBSCAN WITH ESTIMATED DISTANCE

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps = 30)
dbscan.fit(df)
print(dbscan.labels_)
[0 0 0 ... 3 3 3]

plt.scatter(df[‘x‘], df[‘y‘], c = dbscan.labels_)
plt.show()
```

EXAMPLE FOR DBSCAN WITH ESTIMATED DISTANCE

```
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps = 30)
dbscan.fit(df)
print(dbscan.labels_)
[0 0 0 ... 3 3 3]

plt.scatter(df[‘x‘], df[‘y‘], c = dbscan.labels_)
plt.show()
```



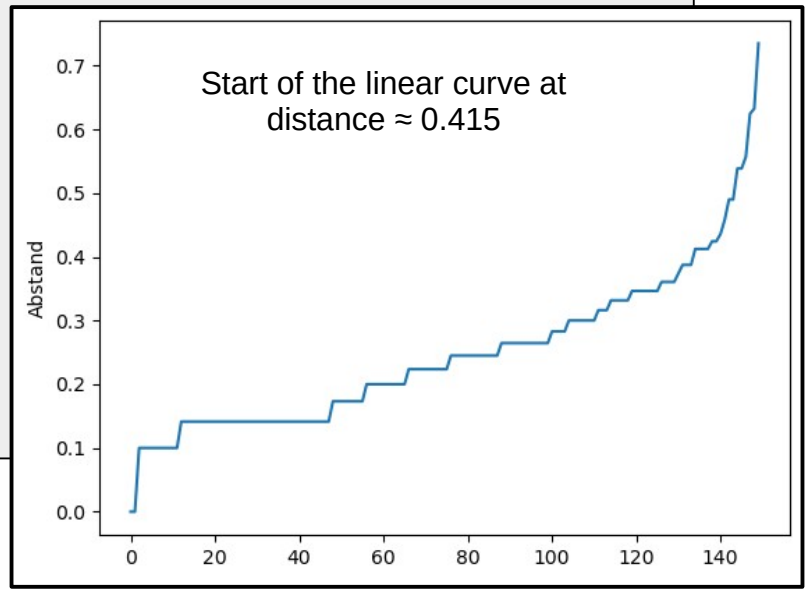
universität wien

EXAMPLE FOR THE IRIS-DATASET

```
x = iris.loc[:,features]

from scipy.spatial.distance import pdist, squareform
dist = pdist(x, "euclidean")
dist = squareform(dist) + 1e100 * np.identity(x.shape[0])
plt.plot(range(x.shape[0]), sorted(dist.min(axis = 0)))
plt.ylabel("Abstand")
plt.show()
```

EXAMPLE FOR THE IRIS-DATASET

```
x = iris.loc[:,features]

from scipy.spatial.distance import pdist, squareform
dist = pdist(x, "euclidean")
dist = squareform(dist) + 1e100 * np.identity(x.shape[0])
plt.plot(range(x.shape[0]), sorted(dist.min(axis = 0)))
plt.ylabel("Abstand")
plt.show()
```



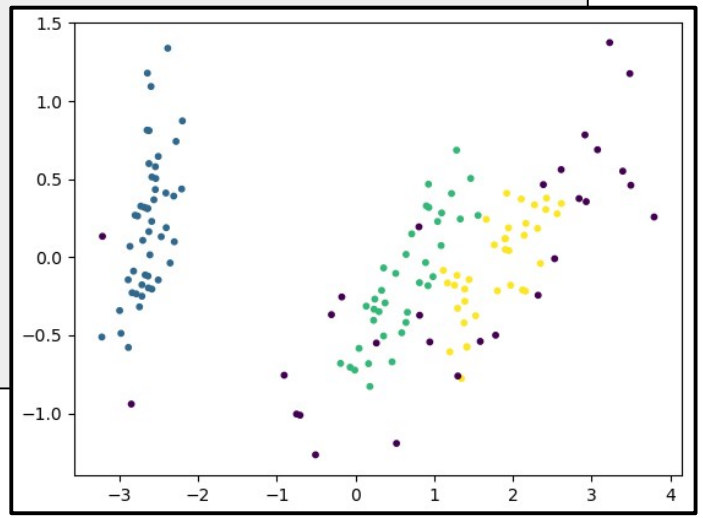
universität wien

EXAMPLE FOR THE IRIS-DATASET

```
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

xt = PCA().fit_transform(x)[:,:2]
dbscan = DBSCAN(eps = 0.415)
dbscan.fit(x)
print(dbscan.labels_)

plt.scatter(xt[:, 0], xt[:, 1], c = dbscan.labels_)
plt.show()
```

EXAMPLE FOR THE IRIS-DATASET

```
from sklearn.cluster import DBSCAN
from sklearn.decomposition import PCA

xt = PCA().fit_transform(x)[:,:2]
dbscan = DBSCAN(eps = 0.415)
dbscan.fit(x)
print(dbscan.labels_)

plt.scatter(xt[:, 0], xt[:, 1], c = dbscan.labels_)
plt.show()
```



universität wien

INTRODUCED CLUSTERING ALGORITHMS

- Hierarchical clustering is suitable, when:
  - we have many dimensions with different variances
  - the cluster number is unknown
  - We want to compare (time-)series with each other (next lecture)

- K-Means is suitable, when:
  - the cluster number is known
  - Cluster sizes and their variance are similar
  - we want a true partitioning

- DBSCAN is suitable, when:
  - the cluster have no trivial geometric form
  - the cluster number is unknown